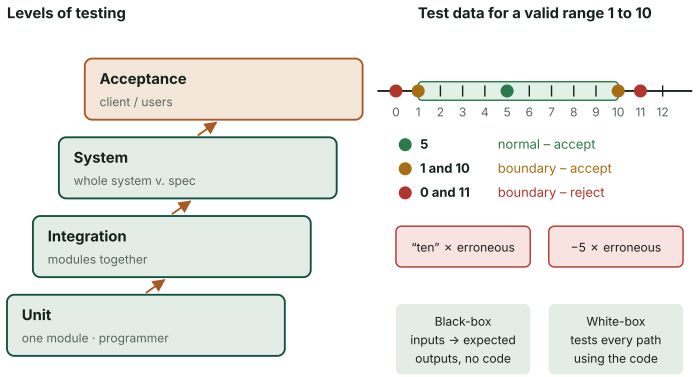
Testing and test plans

AP07 FACT FILE

Specification: students should be able to

- Explain the importance of testing throughout the development of a system in ensuring system quality
- Describe unit and integration testing, including black-box and white-box approaches
- Distinguish between system, acceptance, alpha and beta testing
- Describe the purpose of a test plan and its main components
- Distinguish between normal, boundary (extreme) and erroneous (invalid) test data and select examples for a given rule

Key words: unit testing, integration testing, integration error, system testing, alpha testing, beta testing, acceptance testing, sign-off, black-box testing, white-box testing, test plan, expected outcome, normal data, boundary data, erroneous data



Levels of testing from unit to acceptance, and normal, boundary and erroneous test data on a number line for a valid range of 1 to 10

Overview: testing throughout development

Testing is not a single event at the end of a project but an activity that runs through the whole of development. Software is built as a set of units, so each unit is tested on its own first and any errors are corrected. Units that work perfectly in isolation may still fail when they are joined, so integration testing follows. The complete system is then tested against its agreed specification, the developer carries out a final in-house alpha phase, selected external users may beta test a pre-release version, and finally the client carries out acceptance testing before signing off the system. Testing at every stage matters for quality because an error is far cheaper and easier to fix when it is found early, close to where it was made, than when it is discovered by the client or after the system has gone live. Testing is also planned rather than random: a test plan sets out every test, the data to use and the expected outcome, so that the most vulnerable areas, such as range limits and unusual paths through the code, are always checked. Even thorough testing cannot prove a program is free of errors.

Unit testing: black box and white box

Unit testing checks that each individual component works correctly in all circumstances. Done well, it gives confidence in each unit, which narrows the search when a fault appears later. Two approaches are used. In black-box testing the code is treated as if hidden inside a closed box: the tester knows only what the unit is meant to do, enters inputs and compares the actual outputs with the expected outputs taken from the specification. Testers therefore need no knowledge of the code and can be non-technical. In white-box testing, sometimes called glass-box testing, the tester can see the code and designs tests around its structure, making sure every branch, path and loop is exercised, and concentrating on the places where errors are most likely. For example, a water-billing module that uses one rate for households and two rates for businesses has three branches; black-box tests compare bills for a range of customers, while white-box tests deliberately include at least one case for each branch and cases either side of the 100 cubic metre switch-over point. Programmers often use dry runs and trace tables when white-box testing.

Integration and system testing

Integration testing takes place when units that have already passed their own tests are combined. Faults typically occur at the interfaces between units, because one unit passes data in a form the next does not expect. A classic case is one unit storing dates as day, month, year while another reads them as month, day, year: each unit works and displays sensible dates on its own, but together they misread dates such as the fourth of July as the seventh of April. Such integration errors only appear when the parts are brought together, which is why unit testing alone is never enough. When integration testing finds a fault, the unit concerned is modified, unit tested again and then re-integrated. Agreeing data formats early in a shared data dictionary reduces these problems.

System testing, sometimes called application testing, is then performed on the complete, integrated system to judge whether it complies with the agreed specification. It is essentially black-box testing applied to the whole system rather than one part, so the tester does not need technical knowledge of how it was built, and it is carried out by following a detailed test plan.

Alpha, beta and acceptance testing

Alpha testing is the final phase of in-house testing carried out by the software developer before the software leaves their control. It brings together unit, integration and system testing, follows a test plan and may use simulated or real data; small changes can still be made afterwards. Beta testing comes after alpha testing. A pre-release version is given to a group of potential users who use it in a realistic environment, on their own equipment and without the developers present, and report any faults they find. It can be thought of as external user acceptance testing and is especially common for software that will be sold to many customers or run on many different devices.

Acceptance testing is the final test, carried out once system testing is complete and bespoke software is ready for handover to the client. End users try it in their real working setting, with genuine data at realistic volumes, to gain confidence that it meets their requirements and works alongside the organisation's other systems. They report problems, and once these are resolved the users sign off the software, which formally completes the contract.

Test plans and their components

A test plan is a detailed document that a team of testers follows exactly. Its purpose is to make testing thorough and systematic, so that every requirement, every pathway through the system and every validation routine is checked, and the system's actual performance is compared with its design. For each test the plan records a test number, the part of the system being tested, what is being tested and why (for example, boundary data to check the lowest valid value is accepted), the exact test data and the expected outcome. These columns are completed by the person who designs the tests before any testing happens. The plan is then passed to the tester, who carries out each test and fills in the remaining columns: the actual outcome and PASS or FAIL. A test passes only when the actual outcome matches the expected outcome exactly. Failed tests are reported so the fault can be fixed, after which the tests are run again to make sure the fix works and has not broken anything else. A systems analyst may help by suggesting test data, particularly at the boundaries, and by checking results against the specification.

Choosing test data

Test data falls into three categories. Normal or valid data is the typical data the system expects; it is correct in type and format and should be accepted and processed. For a mark out of 100 stored as a whole number, values such as 43, 72 and 89 are normal. Boundary or extreme data is still valid but lies at the very edges of the allowed range, such as 0 and 100, because this is where programmers most often make off-by-one mistakes, for instance writing less than instead of less than or equal to. Where a range is divided into bands, every band edge is a boundary, so a grading scheme of 0–44, 45–64, 65–79 and 80–100 needs tests at 0, 44, 45, 64, 65, 79, 80 and 100.

Erroneous, invalid or exceptional data is wrong in type, format or range, such as 55.5, the letter X, the word sixty, a negative number or a value above the maximum. It should be rejected, ideally with a helpful message, rather than accepted or causing the program to crash. Testers often also try the values just outside each limit, which are invalid and should be rejected.

T Key terms from the mark schemes

Unit testing
Testing an individual module or subroutine on its own, usually in-house by the programmer, to check it works correctly in all circumstances.

Integration testing
Testing units that have already been tested once they are combined, to check that they interact correctly and pass data between them as expected.

System testing
Testing the complete, integrated system against its agreed specification; essentially black-box testing of the whole system.
Credited: application testing

Alpha testing
The final phase of in-house testing carried out by the developer, following a test plan, before the software is released to users.

Beta testing
Testing of a pre-release version by a group of potential external users in a realistic environment, without developers present, who report any faults.
Credited: external user acceptance testing

Acceptance testing
The final test, by end users in their real working environment with genuine data at realistic volumes, to confirm the system meets their requirements before they sign it off.
Credited: user acceptance testing

Black-box testing
Testing that checks outputs against expected results for given inputs without any access to or knowledge of the program code.
Credited: functional testing

White-box testing
Testing that uses knowledge of the program code to design tests for every path, branch and loop.
Credited: glass-box / structural testing

Test plan
A document listing every test to be carried out, giving the part of the system tested, what is tested, the test data and the expected outcome, with space for the actual outcome and pass/fail.

Normal data
Typical valid data, correct in type and format, that the system should accept and process.
Credited: valid data

Boundary data
Valid test data at the very edges of an allowed range, used to find errors in range limits.
Credited: extreme data

Erroneous data
Test data that is wrong in type, format or range and should be rejected by the system.
Credited: invalid / exceptional data

C C# example

Worked example: a water bill module

```
decimal Bill(decimal m3, bool business)
{
    if (!business)
        return m3 * 1.50m;          // branch 1
    else if (m3 <= 100)
        return m3 * 1.50m;          // branch 2
    else
        return m3 * 1.20m;          // branch 3
}
Console.WriteLine(Bill(10, false));  // 15.00
Console.WriteLine(Bill(100, true));  // 150.00
Console.WriteLine(Bill(101, true));  // 121.20
```

Exam focus

Questions on testing stages usually give one or two marks per stage, so state both what is tested and who tests it or where: unit testing is in-house by the developer on individual components; acceptance testing is by end users in a live environment with real data, ending in sign-off. Do not confuse system testing (the whole system, by the developer, against the specification) with acceptance testing (by the client, against their requirements). For alpha versus beta, contrast in-house developers with external potential users in a realistic environment. Test data questions are common and very precise. For a junior band of 4 to 12 whole years, boundary values are 4 and 12; normal is any integer strictly between; invalid examples include text or a decimal such as 7.5. When a range is open-ended, such as 13 or older, choose a reasonable upper limit and say so. Always give a specific expected outcome, such as an error message stating the valid range, never ‘it works’. A typical slip is to list 0 or 101 as boundary data that should be accepted when they lie outside the valid range.

? Check yourself

1. Explain why testing should take place throughout the development of a system rather than only at the end. [3]
2. Describe what happens during unit testing and integration testing. [4]
3. Explain why a system made of units that have all passed unit testing may still fail integration testing. Use an example. [3]

Answers: 1. Any three from: errors are cheaper/easier to fix when found early, near where they were made (1) / each unit can be checked before being combined, narrowing the search for faults (1) / integration errors only appear when units are joined so must be tested at that stage (1) / the system must be checked against requirements before handover (1) / confidence/quality is built up at each stage (1) 2. Unit: each individual module/component is tested on its own (1) / in-house by the developer/programmer to check it works in all circumstances, using black-box or white-box methods (1). Integration: tested units are combined (1) / to check they work together and data passes correctly between them/at their interfaces (1) 3. Units may expect data in different forms, so faults appear at the interfaces between them (1) / each unit's own tests would not reveal this (1) / example, e.g. one unit stores dates as dd/mm/yyyy and another reads mm/dd/yyyy, so dates are misread when combined (1)